Lecture 15: Transformers

Instructor: Stephanie Schoch

CSCI 680: Natural Language Processing
Department of Computer Science, College of William & Mary
October 12, 2026

WILLIAM & MARY

Many slides are adapted from Jurafsky & Martin (3rd ed.) & Chris Manning.

Lecture Outline

- Recap: Encoder-Decoder & Attention
- Self-Attention
- Multiheaded Attention
- Transformers

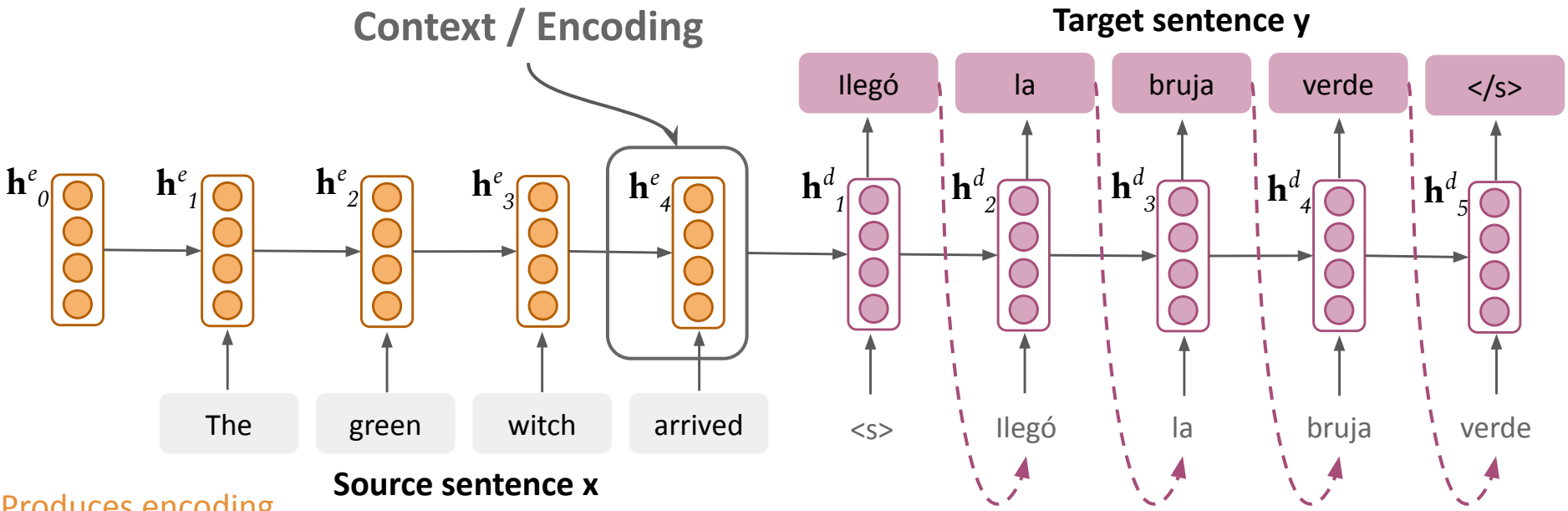
Recap: Encoder-Decoder & Attention

Encoder-Decoder Network

Language Model that produces the target sentence conditioned on the encoding

Represents input sequence.
Provides initial hidden state for Decoder RNN

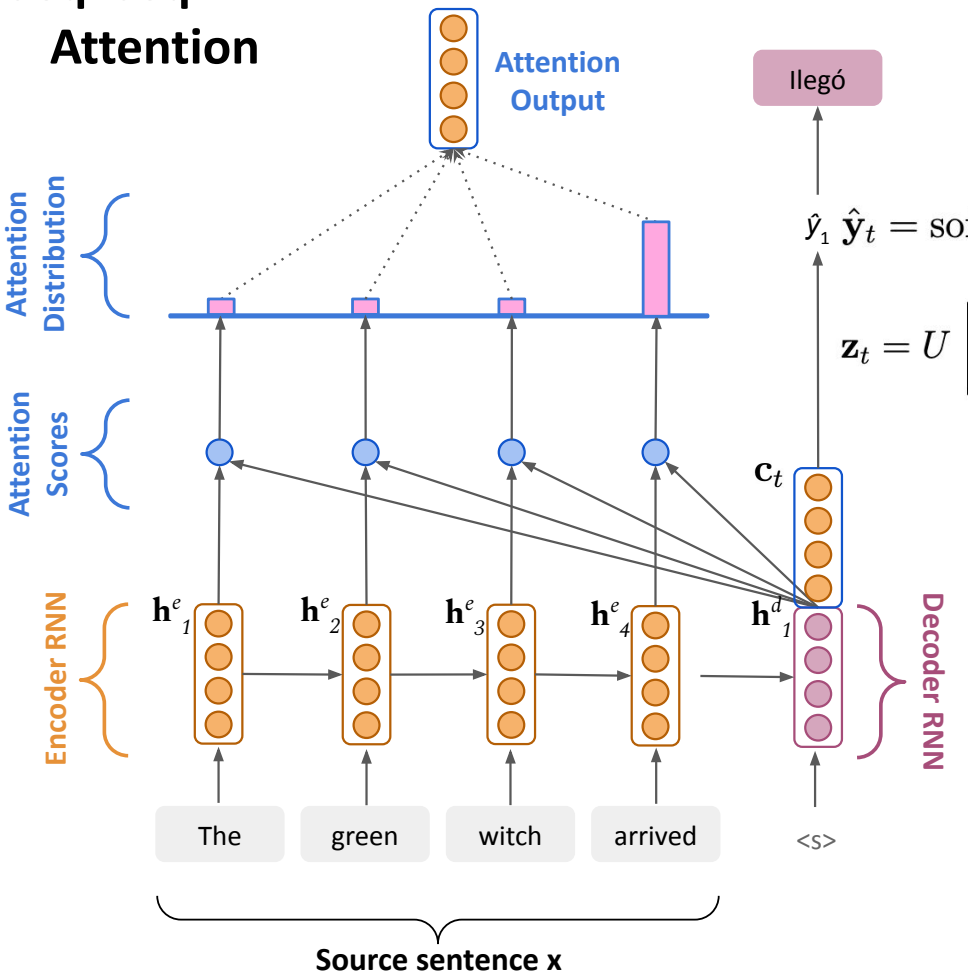
Decoder RNN



Produces encoding of source sentence

Encoder RNN

Seq2seq with Attention

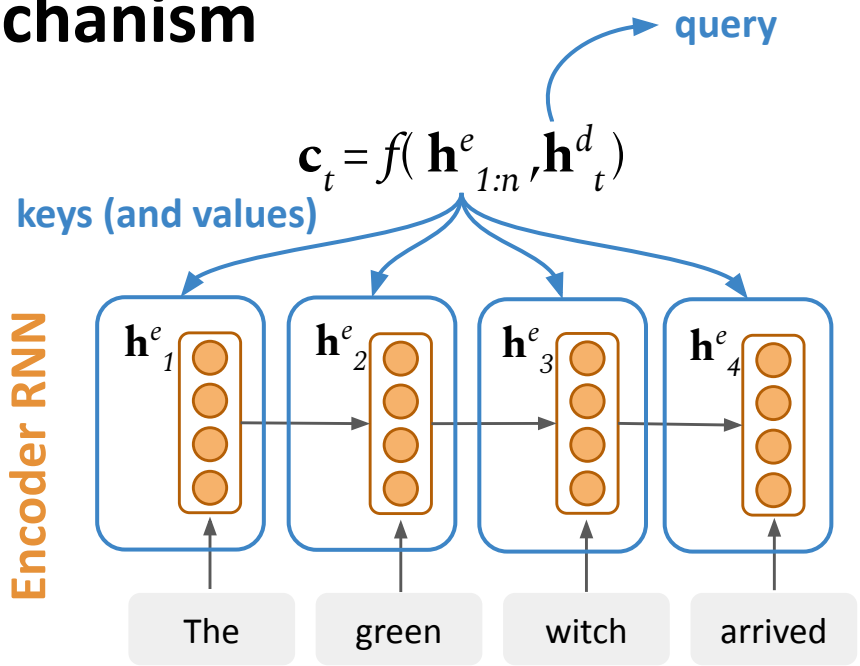


$$U \in \mathbb{R}^{|V| \times 2h}, \qquad \mathbf{b}_2 \in \mathbb{R}^{|V|}, \qquad \mathbf{z}_t \in \mathbb{R}^{|V|}$$

- Attention allows the decoder to focus on a particular part of the source sequence at each time step
- Attention context vector $\mathbf{c}_t$
  - Weighted sum of the encoder hidden states
  - Weights emphasize source information relevant to the next output token (i.e. **attend** to the relevant information)

Attention Mechanism

- In general, we have a single **query** vector and multiple **key** vectors.
  - We want to score each query-key pair
  - **General idea:** attention takes a query, compares it to a set of keys to get attention weights, then uses those weights to combine the corresponding values (in this model, values are encoder hidden states).



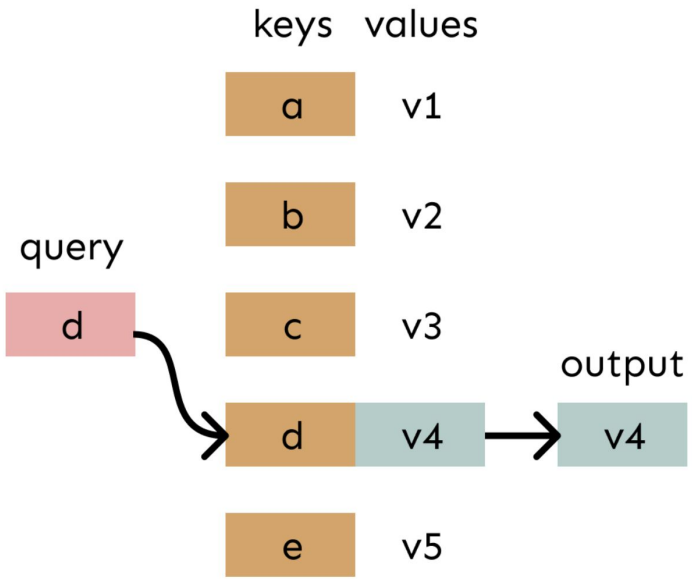
More on Attention

Given a set of vector **values**, and a vector **query**, **attention** is a technique to compute a weighted sum of the values, dependent on the query

- We sometimes say that the query attends to the values.
  - For example, in the seq2seq + attention model, each decoder hidden state (query) *attends* to all the encoder hidden states (values)
  - Keys and values correspond to the same entity (the encoded sequence).
- The weighted sum is a selective summary of the information contained in the values, where the query determines which values to focus on.
- Attention is a way to obtain a fixed-size representation of an arbitrary set of representations (the values), dependent on some other representation (the query).

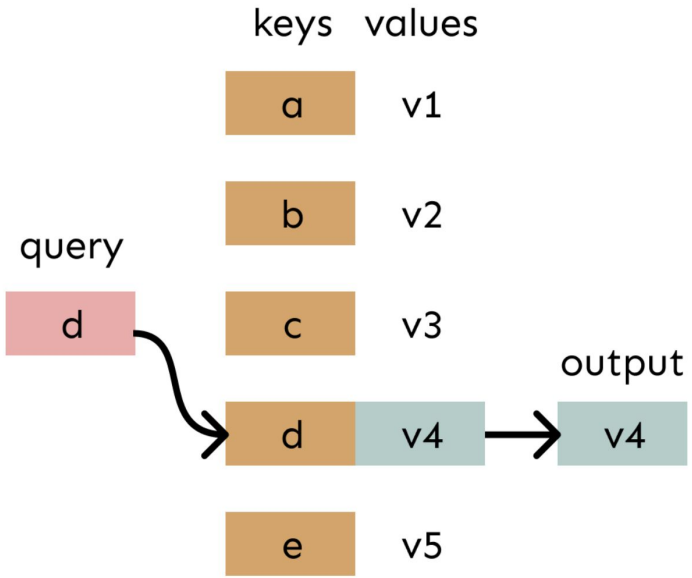
Attention is *weighted* averaging, which lets you do lookups!

In a **lookup table,** we have a table of keys that map to values. The query matches one of the keys, returning its value.

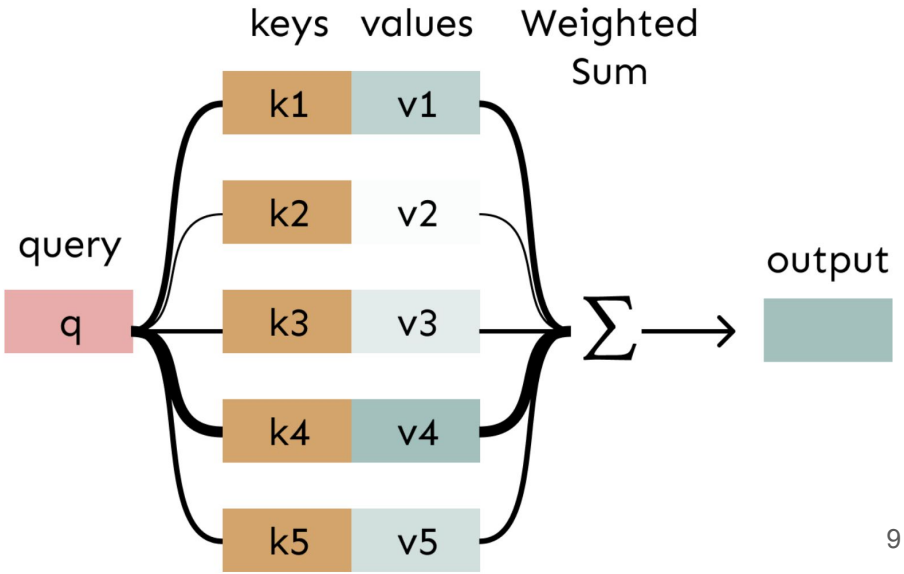


Attention is *weighted* averaging, which lets you do lookups!

In a **lookup table,** we have a table of keys that map to values. The query matches one of the keys, returning its value.



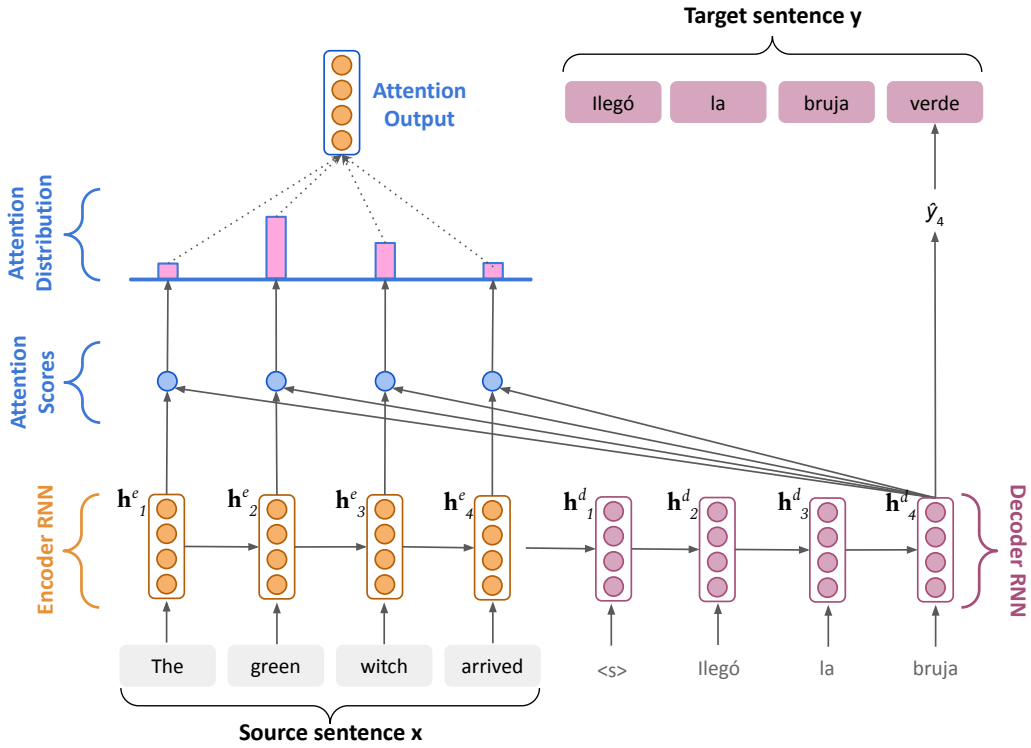
In **attention**, the query matches all keys *softly*, to a weight between 0 and 1. The keys' values are multiplied by the weights and summed.



Do We Need Recurrence?

Abstractly: Attention is a way to pass information from a sequence (x) to a neural network input. (h_t)

- This is also *exactly* what RNNs are used for – to pass information!
- **Can we just get rid of the RNN entirely?** Maybe attention is just a better way to pass information!

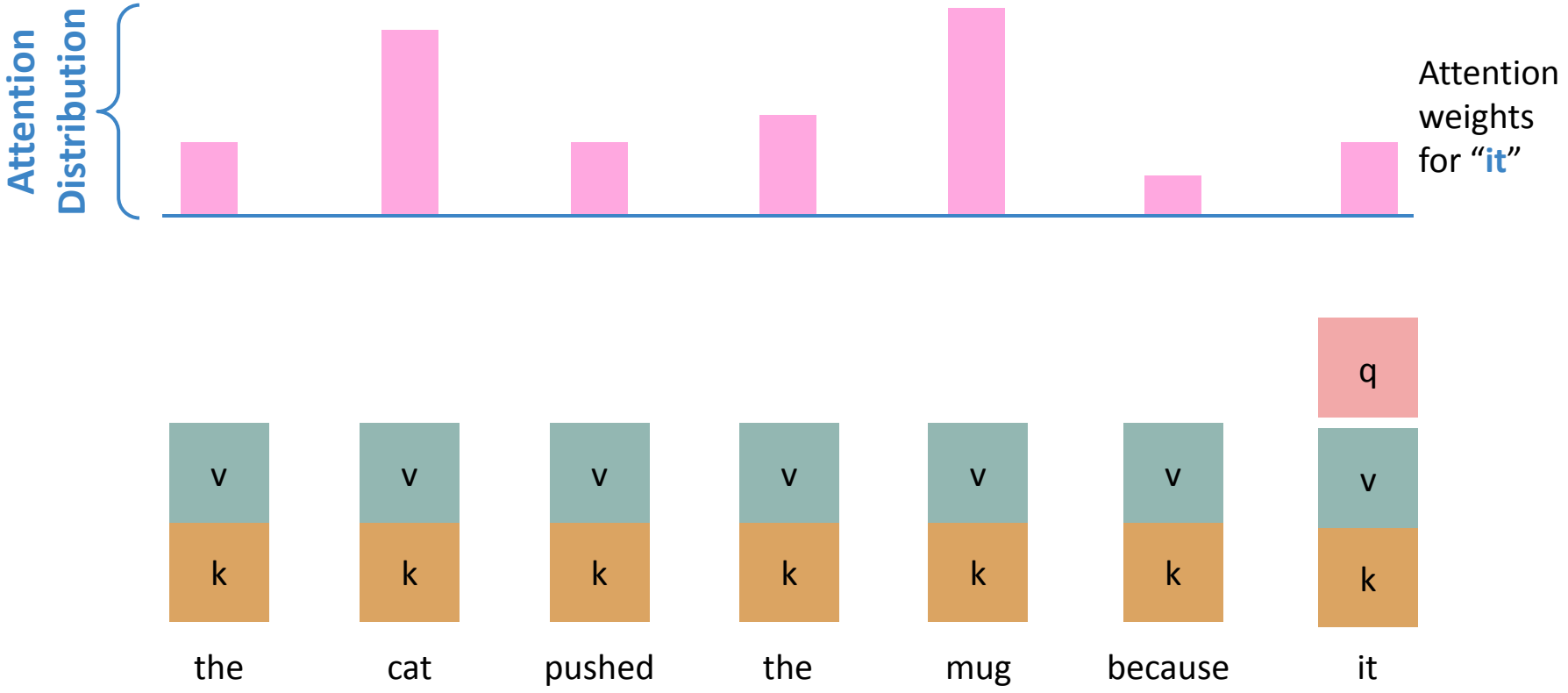


Cross attention: paying attention to the input x to generate y_t

Self attention: to generate y_t, we need to pay attention to $y_{<t}$

Transformers: Self-Attention Networks

Self-Attention: Hypothetical Example

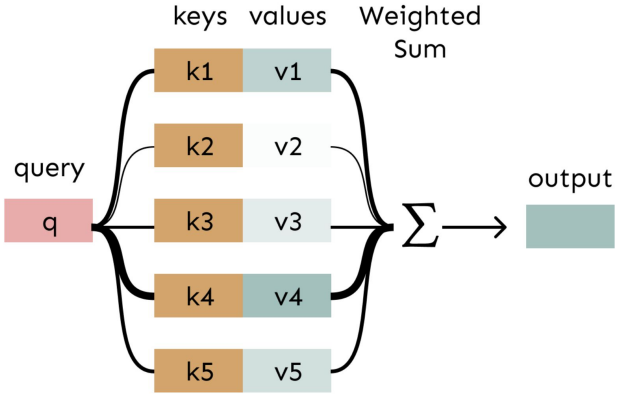


Self-Attention

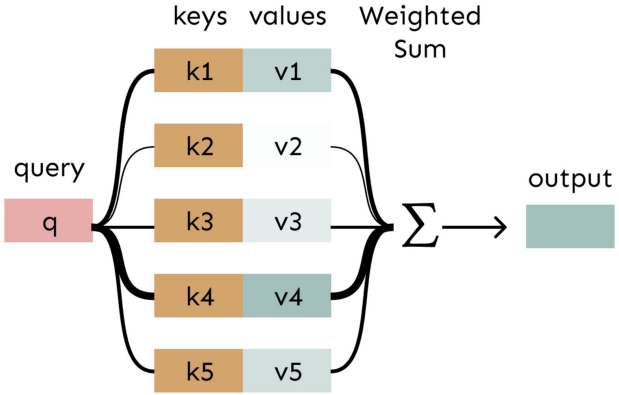
Let $\mathbf{w}_{1:N}$ be a sequence of words in vocabulary V

For each w_i, let $\mathbf{x}_i = \mathbf{E}_{w_i,:} \in \mathbb{R}^{1\times d}$, where

$\mathbf{E} \in \mathbb{R}^{|\mathcal{V}|\times d}$ is an embedding matrix.



Self-Attention



Let $\mathbf{w}_{1:N}$ be a sequence of words in vocabulary V

For each w_i, let $\mathbf{x}_i = \mathbf{E}_{w_i,:} \in \mathbb{R}^{1\times d}$, where

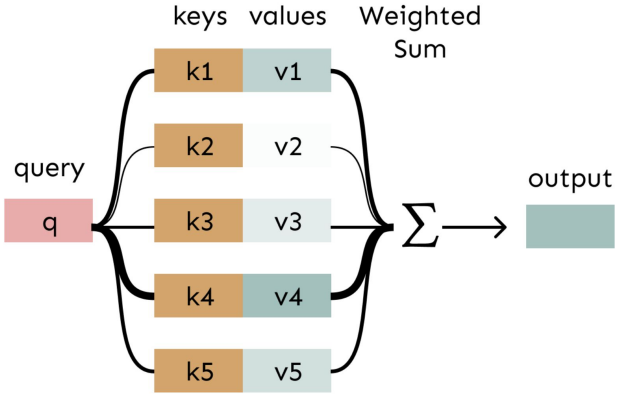
$\mathbf{E} \in \mathbb{R}^{|\mathcal{V}|\times d}$ is an embedding matrix.

1. Transform each word embedding with weight matrices $\mathbf{W}^Q$, $\mathbf{W}^K$, $\mathbf{W}^V$

$$\mathbf{W}^Q \in \mathbb{R}^{d\times d_k} \qquad \mathbf{W}^K \in \mathbb{R}^{d\times d_k} \qquad \mathbf{W}^V \in \mathbb{R}^{d\times d_v}$$

$$\mathbf{q}_i = \mathbf{x}_i\mathbf{W}^Q \in \mathbb{R}^{1\times d_k} \qquad \mathbf{k}_i = \mathbf{x}_i\mathbf{W}^K \in \mathbb{R}^{1\times d_k} \qquad \mathbf{v}_i = \mathbf{x}_i\mathbf{W}^V \in \mathbb{R}^{1\times d_v}$$

Self-Attention



Let $\mathbf{w}_{1:N}$ be a sequence of words in vocabulary V

For each w_i, let $\mathbf{x}_i = \mathbf{E}_{w_i,:} \in \mathbb{R}^{1\times d}$, where

$\mathbf{E} \in \mathbb{R}^{|\mathcal{V}|\times d}$ is an embedding matrix.

1. Transform each word embedding with weight matrices $\mathbf{W}^Q$, $\mathbf{W}^K$, $\mathbf{W}^V$

$$\mathbf{W}^Q \in \mathbb{R}^{d\times d_k} \qquad \mathbf{W}^K \in \mathbb{R}^{d\times d_k} \qquad \mathbf{W}^V \in \mathbb{R}^{d\times d_v}$$

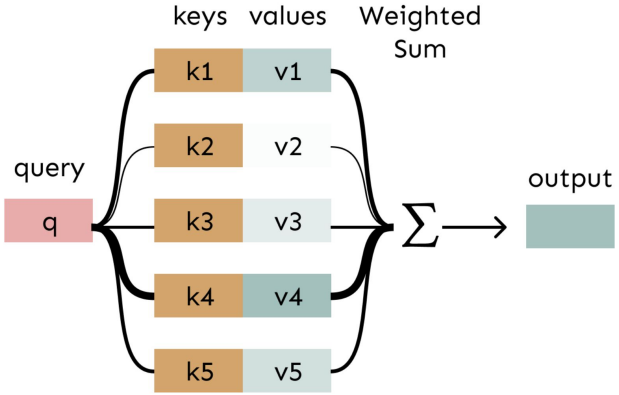
$$\mathbf{q}_i = \mathbf{x}_i\mathbf{W}^Q \in \mathbb{R}^{1\times d_k} \qquad \mathbf{k}_i = \mathbf{x}_i\mathbf{W}^K \in \mathbb{R}^{1\times d_k} \qquad \mathbf{v}_i = \mathbf{x}_i\mathbf{W}^V \in \mathbb{R}^{1\times d_v}$$

2. Compute similarities between keys and queries; normalize with softmax

$$e_{ij} = \frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d_k}} \qquad \alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{j'=1}^{N}\exp(e_{ij'})}$$

3.

Self-Attention



Let $\mathbf{w}_{1:N}$ be a sequence of words in vocabulary V

For each w_i, let $\mathbf{x}_i = \mathbf{E}_{w_i,:} \in \mathbb{R}^{1\times d}$, where

$\mathbf{E} \in \mathbb{R}^{|\mathcal{V}|\times d}$ is an embedding matrix.

1. Transform each word embedding with weight matrices $\mathbf{W}^Q$, $\mathbf{W}^K$, $\mathbf{W}^V$

$$\mathbf{W}^Q \in \mathbb{R}^{d\times d_k} \qquad \mathbf{W}^K \in \mathbb{R}^{d\times d_k} \qquad \mathbf{W}^V \in \mathbb{R}^{d\times d_v}$$

$$\mathbf{q}_i = \mathbf{x}_i\mathbf{W}^Q \in \mathbb{R}^{1\times d_k} \qquad \mathbf{k}_i = \mathbf{x}_i\mathbf{W}^K \in \mathbb{R}^{1\times d_k} \qquad \mathbf{v}_i = \mathbf{x}_i\mathbf{W}^V \in \mathbb{R}^{1\times d_v}$$

2. Compute similarities between keys and queries; normalize with softmax

$$e_{ij} = \frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d_k}} \qquad \alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{j'=1}^{N}\exp(e_{ij'})}$$

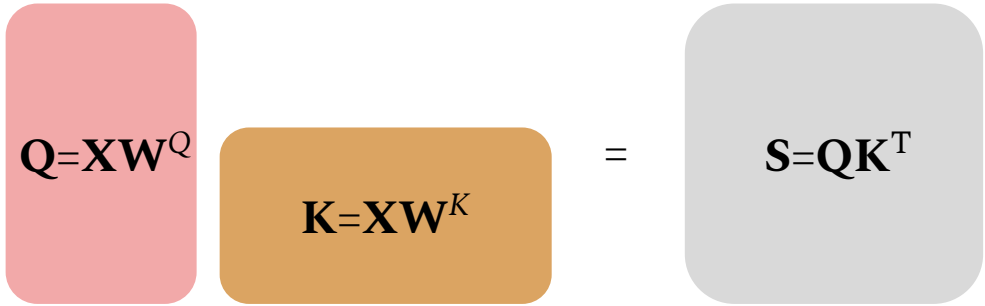
3. Compute output for each word as weighted sum of values: $\mathbf{o}_i = \sum_{j=1}^{N}\alpha_{ij}\mathbf{v}_j \in \mathbb{R}^{1\times d_v}$

Self-Attention: Matrix Multiplications

- Key-query-value attention is typically computed as matrices.
- Let $\mathbf{X} = \begin{bmatrix} \mathbf{x}_1 \\ \vdots \\ \mathbf{x}_N \end{bmatrix} \in \mathbb{R}^{N \times d}$ be the concatenation of input vectors
- Query, Key, Value matrices: $\mathbf{Q} = \mathbf{X}\mathbf{W}^Q \in \mathbb{R}^{N \times d_k}$, $\mathbf{K} = \mathbf{X}\mathbf{W}^K \in \mathbb{R}^{N \times d_k}$, $\mathbf{V} = \mathbf{X}\mathbf{W}^V \in \mathbb{R}^{N \times d_v}$
- Score matrix: $\mathbf{S} = \mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{N \times N}$
- Attention matrix: $\mathbf{A} = \mathrm{softmax}_{\mathrm{row}}\left(\frac{\mathbf{S}}{\sqrt{d_k}}\right)$
- Output matrix: $\mathbf{O} = \mathbf{A}\mathbf{V} \in \mathbb{R}^{N \times d_v}$

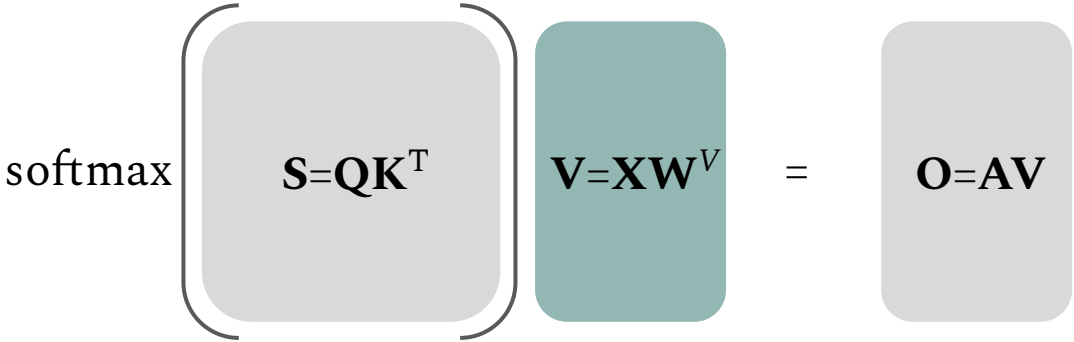
Self-Attention: Matrix Multiplications

First, take the query-key dot products in one matrix multiplication

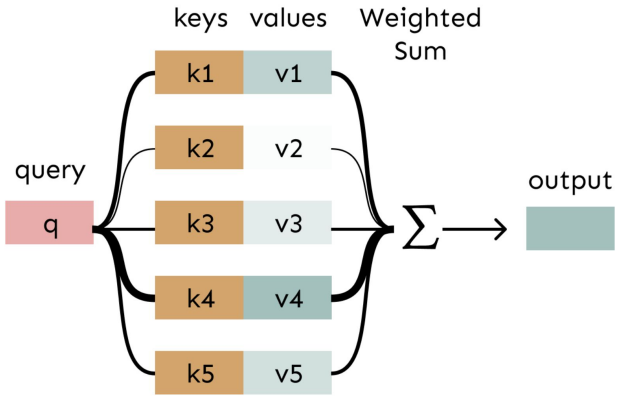


All pairs of attention scores!

Next, softmax, and compute the weighted average with the values (with another matrix multiplication)



Why Self-Attention?



- Self-attention allows a network to directly extract and use information from arbitrarily large contexts without the need to pass it through intermediate recurrent connections as in RNNs
- Used often with feedforward networks!

Transformers are Self-Attention Networks

- Self-Attention is the key innovation behind Transformers!
- Transformers map sequences of input vectors $\mathbf{x}_{1:N}$ to sequences of output vectors $\mathbf{y}_{1:N}$ of the same length.
- Made up of stacks of Transformer blocks
  - each of which is a multilayer network made by combining
    - simple linear layers,
    - feedforward networks, and
    - self-attention layers
- No recurrent connections!



Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com

Noam Shazeer* Google Brain noam@google.com

Niki Parmar* Google Research nikip@google.com

Jakob Uszkoreit* Google Research usz@google.com

Llion Jones* Google Research llion@google.com

Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu

Łukasz Kaiser* Google Brain lukaszkaiser@google.com

Illia Polosukhin* ‡ illia.polosukhin@gmail.com

31st Conference on Neural Information Processing Systems (NIPS 2017), Long Beach, CA, USA.

TODOs:

- **Homework 2 Due: 10/23**
- **Quiz 2: 10/14 (Wednesday)**

Next Class: Transformers (II)